



Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Proceso de Desarrollo Software TPO

Profesora: Leon María Angela

Equipo: "Proyecto Y"

Integrantes:

Perez Leal, Agustín - 1189636

Martinez Zorzi, Facundo - 1193730

Mioni Bonuccelli, Franco Brunello - 1189642

Smith, Pedro - 1195404

1. Descripción general de la problemática

La empresa viene trabajando en un simulador de fútbol pero el sistema que tienen no aguanta el crecimiento. Cada vez que quieren agregar una táctica nueva o un tipo de evento, terminan tocando clases centrales del motor y rompiendo algo en otro lado. El código no tiene una estructura clara y ahora cualquier cambio es una cadena de efectos no deseados.

El problema más concreto es el acoplamiento entre el motor de simulación y los módulos que lo rodean. Por ejemplo, el marcador, el relato deportivo y las estadísticas están conectados directamente al motor: si querés sacar uno o agregar otro, tenés que meterte en el núcleo del sistema, lo que nos lleva al acoplamiento. Esto no debería ser así.

Tampoco hay un modelo claro para el ciclo de vida de un partido. En qué momento se puede hacer un cambio de jugador, cuándo se habilitan los penales, qué pasa en el entretiempo.

A eso se le suma que el sistema no guarda nada entre sesiones. Los equipos, los jugadores, los resultados, todo desaparece al cerrar. No hay forma de consultar el historial de un torneo ni retomar algo que quedó a medias.

Por último, no hay una interfaz visual. La configuración del partido, el seguimiento del marcador y el relato en tiempo real, las estadísticas, todo eso hoy no tiene una presentación accesible para el usuario.

La propuesta es construir el sistema desde cero en Java, con una interfaz visual y persistencia, aplicando patrones de diseño que resuelvan cada uno de estos problemas.

2. Objetivos Del Sistema

1. **Gestionar equipos y jugadores:** que el sistema permite registrar, modificar y consultar equipos con sus jugadores, posiciones, estado físico y táctica asignada. Todo con persistencia en base de datos para no perder nada entre sesiones.
2. **Simular partidos:** un motor que genere eventos deportivos reales: goles, faltas, tarjetas, lesiones, penales. La probabilidad de cada evento varía según la táctica que tenga activa cada equipo en ese momento.
3. **Modelar el ciclo de vida del partido:** el partido tiene estados concretos: primer tiempo, entretiempo, segundo tiempo, finalizado. Cada estado define qué se puede hacer y qué no. Un cambio de jugador en el entretiempo, por ejemplo, no debería ser posible.
4. **Desacoplar los módulos de salida del motor:** cuando ocurre un evento, el marcador, las estadísticas y el relato deportivo se tienen que enterar solos. El motor no debería saber quién está escuchando ni cuántos son.

5. **Cambiar la táctica durante el partido:** el técnico puede modificar la estrategia de su equipo en cualquier momento del encuentro sin tocar ni interrumpir la lógica del motor.
6. **Persistir y recuperar datos:** equipos, jugadores, partidos y resultados se guardan en la base de datos y se recuperan al iniciar el sistema. El historial tiene que estar disponible siempre.
7. **Interfaz visual funcional:** una UI donde el usuario pueda configurar un partido, seguir el marcador y el relato en tiempo real, ver las estadísticas y consultar los datos guardados.
8. **Diseño extensible:** agregar una táctica nueva, un tipo de evento o un modo de juego distinto no debería requerir tocar las clases que ya funcionan.

3. Alcance del funcional

Qué incluye:

- Registro y administración de equipos, jugadores y estadios.
- Configuración de partidos: selección de equipos, táctica inicial y modo de juego (amistoso o torneo).
- Simulación completa de un partido dividida en tramos, primer tiempo, entretiempo y segundo tiempo, con generación de eventos en cada uno.
- Cambio de táctica durante el partido desde la interfaz, en cualquier momento.
- Visualización en tiempo real del marcador, el relato evento por evento y las estadísticas del partido.
- Persistencia de equipos, jugadores y resultados en base de datos.
- Consulta del historial de partidos jugados.

Qué no incluye:

- Renderizado gráfico del campo ni animaciones de ningún tipo
- IA autónoma por jugador — las decisiones son del motor probabilístico, no de agentes individuales
- Modo carrera, mercado de pases o gestión económica de clubes
- Integración con APIs externas de datos futbolísticos reales
- Autenticación de usuarios ni manejo de roles

4. Requisitos Funcionales

Gestión de equipos y jugadores

- RF01: Se puede registrar un equipo con nombre, estadio y plantilla de jugadores.
- RF02: Se puede registrar un jugador con nombre, posición, estado físico (disponible, lesionado o suspendido) y equipo al que pertenece.
- RF03: Los datos de equipos y jugadores se pueden modificar y consultar.
- RF04: Antes de cada partido se puede asignar una táctica al equipo.

Configuración del partido

- RF05: Se puede configurar un partido eligiendo dos equipos, estadio, duración y modo de juego (amistoso o torneo).
- RF06: El partido se puede iniciar, pausar y finalizar desde la interfaz.

Simulación

- RF07: El motor genera eventos durante el partido: goles, faltas, tarjetas amarillas, tarjetas rojas, lesiones y penales.
- RF08: La probabilidad de cada evento varía según la táctica activa del equipo en ese momento.
- RF09: La táctica de un equipo se puede cambiar en cualquier momento del partido.
- RF10: Cada evento registra el minuto en que ocurrió y el jugador involucrado.
- RF11: El marcador se actualiza solo ante cada gol o penal convertido.
- RF12: Las estadísticas acumulan goles, faltas y tarjetas por equipo y por jugador.
- RF13: El relato genera una línea descriptiva por cada evento, en el momento en que ocurre.

Persistencia e historial

- RF14: Equipos, jugadores y resultados se guardan en la base de datos automáticamente.
- RF15: Se puede consultar el historial de partidos jugados.

5. Requisitos no funcionales

- RNF01: El sistema está implementado en Java con programación orientada a objetos.
- RNF02: El sistema contará con una interfaz visual.
- RNF03: La persistencia usa bases de datos para almacenar la información.
- RNF04: El proyecto gestiona versiones entre los integrantes mediante GitHub.
- RNF05: La arquitectura está organizada en paquetes con separación clara entre dominio, servicios y presentación.
- RNF06: El sistema corre en cualquier máquina con Java 17 instalado, sin necesidad de configurar ningún servidor.
- RNF07: Agregar una táctica nueva o un tipo de evento distinto no requiere tocar las clases que ya funcionan.

6. Diagrama de clases

Se adjuntó en la entrega junto con los demás archivos correspondientes.

7. Identificación de actores o usuarios

En el sistema tendremos solo un actor principal, el cual será el **usuario (jugador)** que va a interactuar con el sistema a través de la interfaz visual del mismo. Todas las funcionalidades del sistema son accesibles por este actor, ya que no hay roles con permisos diferenciados.

Algunas de las funciones que va a poder ejecutar este actor son:

- Registrar y administrar equipos y jugadores.
- Configurar partidos.
- Iniciar y controlar la simulación.
- Cambiar la táctica durante el partido.
- Consultar marcador, estadísticas y relato en tiempo real.
- Consultar el historial de partidos jugados.

Luego como actores secundarios tendremos al Motor de Simulación dentro del sistema y la Base de Datos por fuera de este.

El Motor de Simulación se va a encargar de generar los eventos dentro de la simulación, no es una persona como lo es el usuario pero si es un actor que genera interacciones con otros módulos del sistema.

La Base de Datos es la encargada de recibir y proveer datos necesarios sobre el sistema y será un actor con el que interactuara el videojuego.

8. Casos de uso principales

CU01: Registrar equipo El usuario ingresa el nombre del equipo y su estadio. El sistema valida los datos, los guarda en la base de datos y confirma el registro.

CU02: Registrar jugador Con al menos un equipo existente, el usuario ingresa nombre, posición y equipo al que pertenece. El sistema lo guarda con estado DISPONIBLE por defecto y confirma.

CU03: Configurar partido El usuario elige equipo local, visitante, estadio, duración y modo de juego (amistoso o torneo). Asigna una táctica inicial a cada equipo. El sistema construye el partido a través del "ConstructorPartido" y confirma la configuración.

CU04: Iniciar simulación El usuario presiona iniciar. El sistema pasa al estado "PrimerTiempo" y el motor empieza a generar eventos. Cada evento se envía automáticamente al marcador, las estadísticas y el relato. Al terminar el tramo, el sistema transiciona a Entretiempo.

CU05: Cambiar táctica Durante el partido (PrimerTiempo o SegundoTiempo), el usuario selecciona un equipo y una táctica nueva. El sistema la actualiza y el motor la usa para los siguientes eventos generados.

CU06: Consultar marcador y estadísticas Desde el panel de juego, el usuario puede ver en cualquier momento el marcador actualizado, las estadísticas acumuladas por equipo y jugador, y el relato con todos los eventos ocurridos hasta ese momento.

CU07: Finalizar partido Al completarse el segundo tiempo, el sistema cambia el estado a Finalizado, guarda el resultado en la base de datos y muestra el resumen final con estadísticas completas.

CU08: Consultar historial El usuario accede a la sección de historial. El sistema consulta la base de datos y muestra la lista de partidos jugados con fecha, equipos y resultado.

9. Diagramas de caso de uso

Adjuntos en la entrega con los demás archivos correspondientes.

10. Primer listado de clases candidatas.

Las clases identificadas hasta el momento son las siguientes, están divididas por paquetes y por patrones:

Modelo

Son las clases que representan el mundo del sistema. Sin estas no hay nada.

Clase	Tipo
Jugador	Clase
Equipo	Clase
Estadio	Clase
Partido	Clase
EventoDeportivo	Clase
EstadoJugador	Enum
TipoEvento	Enum

Táctica, Patrón Strategy

Cada táctica es una clase separada que implementa la misma interfaz. Cambiar de táctica es solo cambiar el objeto.

Clase	Tipo
ITactica	Interface
TacticaOfensiva	Clase
TacticaDefensiva	Clase
TacticaEquilibrada	Clase

Estado, Patrón State

El partido sabe en qué momento está y qué se puede hacer en cada uno.

Clase	Tipo
IEstadoPartido	Interface
EstadoPrimerTiempo	Clase
EstadoEntretiempo	Clase
EstadoSegundoTiempo	Clase
EstadoFinalizado	Clase

Observador, Patrón Observer

Cuando el motor genera un evento, estos tres se enteran solos. El motor no los conoce directamente.

Clase	Tipo
IObservadorPartido	Interface
Marcador	Clase
Estadisticas	Clase
RelatoDeportivo	Clase

Simulación, Motor

El corazón del sistema. Genera los eventos y los manda.

Clase	Tipo
MotorSimulacion	Clase

Builder, Patrón Builder

Armar un partido tiene muchas configuraciones opcionales. El builder lo resuelve limpio.

Clase	Tipo
ConstructorPartido	Clase

Fachada, Patrón Facade

La UI habla solo con esta clase. Todo lo demás queda detrás.

Clase	Tipo
ControladorPartido	Clase

Persistencia, Acceso a datos

Todo lo que toca la base de datos vive acá. El resto del sistema no sabe que SQLite existe.

Clase	Tipo
ConexionDB	Clase (Singleton)
EquipoDAO	Clase
JugadorDAO	Clase
PartidoDAO	Clase

11. Repositorio Git

Se adjunto el Link del Repositorio de nuestro proyecto en GitHub:

<https://github.com/FrancoxHCK/TPODDS.git>